پروژه‌ی Paradiso — بک‌اند (NestJS)

فیلتر محتوای بزرگسال

چطور فیلم‌ها و افراد نامناسب را تشخیص می‌دهیم

و در کدام بخش‌ها اعمال می‌کنیم

۹ آبان ۱۴۰۵ (۳۱ اکتبر ۲۰۲۶)

---

۱. خلاصه در یک نگاه

سه بخش با هم کار می‌کنند:

بخش	چیست	فایل
قانون‌ها	توابع خالص که از روی اطلاعات TMDb (برچسب، متن، کلیدواژه، استودیو، تعداد رأی) تصمیم می‌گیرند	`src/movies/content-filter.ts`
لیست مسدودی	جدول `content_block`: فیلم‌ها و افرادی که با شناسه‌ی TMDb برای همیشه ممنوع‌اند	`content-block.entity.ts` `content-blocks.service.ts`
ممیزی (Audit)	اسکریپتی که کتابخانه‌ی فعلی را می‌گردد، قانون‌ها را اجرا می‌کند و لیست مسدودی را پر می‌کند	`scripts/audit-adult-content.cjs` `src/movies/content-audit.ts`

> **اصل طراحی**
>
> **سخت‌گیر عمداً.** برچسب «adult» در TMDb خیلی چیزها را از قلم می‌اندازد (فیلم‌های نیمه‌برهنه، استریپ‌کلاب، مجموعه‌های برهنگی). پس برچسب را با متن و کلیدواژه و استودیو ترکیب می‌کنیم. هدف «داده‌ی تمیز» است، نه «کامل بودن»؛ بنابراین بعضی فیلم‌ها و افراد بی‌گناه هم مسدود می‌شوند.

۲. چطور تشخیص می‌دهیم

فیلم — تابع `adultContentReason`

قانون‌ها به ترتیب اجرا می‌شوند و اولین قانونی که نقض شود دلیل مسدودی است (برای گزارش ممیزی ذخیره می‌شود).

#	قانون	چه چیزی را می‌گیرد	چه وقت اعمال می‌شود
۱	برچسب TMDb	`adult = true`	همیشه
۲	کلمات هسته‌ای	در عنوان، خلاصه‌ی داستان یا کلیدواژه‌ها: `hentai`، `softcore`، `erotic*`، `sexploitation`، `cam girls` و…	همیشه، هر قدر معروف
۳	کلمات صریح	`naughty`، `lust*`، `bikini`، `playboy`، `centerfold`، `beautiful model` و مانند آن	فقط زیر ۳۰۰۰ رأی
۴	استودیوی بزرگسال	Playboy، Penthouse، Vivid، Brazzers، Hustler و…	همیشه
۵	خط «معروف بودن»	اگر فیلم حداقل ۳۰۰ رأی داشته باشد، از اینجا به بعد قبول می‌شود	—
۶	کلمات تحریک‌آمیز	`porn*`، `xxx`، `stripper`، `nude`، `sexy`، `models`، `seduc*` و…	فقط زیر ۳۰۰ رأی (گمنام)

> **چرا تعداد رأی؟**
>
> فیلم‌های معروف گاهی همین کلمات را دارند و بی‌گناه‌اند (مثل *Killers of the Flower Moon* با عبارت `lust for wealth`). دو آستانه داریم: `OBSCURE_VOTE_COUNT = 300` و `MAINSTREAM_VOTE_COUNT = 3000`.

فرد — تابع `adultPersonReason`

- برچسب `adult` در TMDb ← مسدود.
- متن **بیوگرافی** با فهرست کلمات بررسی می‌شود. عبارت «role model» قبلاً حذف می‌شود تا اشتباه نشود.

- **دو سطح:** فردی که فقط فیلم‌های گمنام دارد با فهرست کامل (شامل sexy، magazine، modeling و مانند آن) سنجیده می‌شود. فردی که حداقل یک فیلم با ۳۰۰۰ رأی دارد فقط با کلمات صنعت بزرگسال (porn، onlyfans، playmate، erotic و مانند آن).

موارد کوچک دیگر

- `BLOCKED_IMAGE_PATHS` : تصویرهای مشخصی (مثلاً یک backdrop نامناسب) هرگز ذخیره یا نشان داده نمی‌شوند.
- `isArchiveFootage` : نقش‌های «Herself (archive footage)» از بازیگران فیلم حذف می‌شوند.

۳. کجا اعمال می‌شود

آیتم‌های تازه از TMDb (هنوز در سیستم ما نیستند)

هر مرحله سخت‌گیرتر از قبلی است و ورود (import) سخت‌ترین است، چون اطلاعات کامل (کلیدواژه و استودیو) را دارد.



نقطه	چه اتفاقی می‌افتد	نتیجه
جستجوی فیلم `GET /movies/tmdb/search`	درخواست با `include_adult=false` به TMDb می‌رود. سپس `isAdultContent` روی هر نتیجه (نتایج جستجو کلیدواژه و استودیو ندارند). بعد برای هر نتیجه‌ی باقی‌مانده یک **جستجوی کلیدواژه** می‌شود که اگر «erotic» داشته باشد حذف شود (`isErotic` ، در صورت خطا بسته می‌ماند). در آخر کنترلر نتایج مسدودشده را حذف می‌کند.	آیتم از لیست حذف می‌شود
جزئیات فیلم `GET /movies/tmdb/:id`	اول لیست مسدودی، بعد `isAdultContent`	۴۰۴ `Movie not found in catalogue`

ورود فیلم `POST /movies/resolve`	لیست مسدودی؛ بعد جزئیات کامل با کلیدواژه و استودیو گرفته می‌شود و `isAdultContent` اجرا می‌شود	۴۲۲ `This title is not available`
ساخت فیلم `POST /movies`	`isAdultContent` روی بدنه‌ی درخواست + لیست مسدودی	۴۲۲ همان پیام
کارت فیلم در چت اتاق	برای شناسه‌های ناشناخته: لیست مسدودی، بعد جزئیات TMDb و `isAdultContent`	۴۲۲ همان پیام
صفحه‌ی فرد `GET /people/:id`	لیست مسدودی؛ `isAdultPerson` روی بیوگرافی (و بیوگرافی انگلیسی)؛ فیلم‌های نامناسب از فیلموگرافی حذف می‌شوند (`isAdultContent` برای هر اثر)؛ آثار مسدودشده هر بار از لیست حذف می‌شوند	۴۰۴ `Person not found`
جستجو و محبوب‌ترین افراد (مدیر / انتخاب هنرمند روز)	`include_adult=false` به TMDb، و حذف هر فردی که `adult` دارد	فرد از لیست حذف می‌شود

آیتم‌های موجود در کتابخانه‌ی ما

مکانیزم	چطور کار می‌کند
ممیزی آفلاین `audit-adult-content.cjs`	همه‌ی فیلم‌های کتابخانه و ۱۵ بازیگر اول هر فیلم را بررسی می‌کند. **پیش‌نمایش** گزارش `audit/report.json` را می‌نویسد و چیزی را تغییر نمی‌دهد. با `--apply` ردیف‌ها را در `content_block` می‌نویسد و فیلم‌های مسدودشده را از کتابخانه **حذف** می‌کند (با فایل پشتیبان). پاسخ‌های TMDb در `audit/tmdb-cache.json` ذخیره می‌شود، پس اجرای مجدد بعد از تغییر قانون ارزان است.
پیوندها (Linked)	ممیزی چند دور پیوندها را دنبال می‌کند: فرد مسدودشده ← فیلم‌های گمنام او (زیر ۳۰۰ رأی) مسدود می‌شوند. فیلم مسدودشده ← بازیگرانی که هیچ فیلم معروفی (حداقل ۵۰۰ رأی) ندارند مسدود می‌شوند.
دستی و استثنا	فهرست‌های داخل اسکریپت: `MANUAL_*` (گزارش‌شده، همیشه مسدود) و `KEEP_*` (حفظ دستی؛ مثلاً *Three Colors: Red*، هرگز مسدود نمی‌شود).
صفحه‌ی جزئیات فیلم `GET /movies/:id/details`	در زمان خواندن، افراد مسدودشده از بازیگران، کارگردان‌ها و عوامل حذف می‌شوند و تصویرهای مسدودشده برداشته می‌شوند. ردیف ذخیره‌شده دست‌نخورده می‌ماند، پس رفع مسدودی نیاز به به‌روزرسانی ندارد.

انتخاب‌های سردبیر (Curations)	تابع `visible()` فیلم‌های مسدودشده را از فهرست حذف می‌کند.

> **نکته‌ی مهم درباره‌ی کتابخانه**
>
> لیست فیلم‌های کتابخانه (`GET /movies` و `GET /movies/:id`) هنگام خواندن فیلتر مسدودی ندارد. فیلمی که بعداً به `content_block` اضافه شود تا اجرای ممیزی با `--apply` در کتابخانه می‌ماند. جلوگیری از ورود تازه‌ها با دروازه‌های بخش قبل انجام می‌شود.

۴. جدول `content_block`

ستون	معنی
`kind`	`movie` یا `person`
`tmdbId`	شناسه‌ی TMDb (نه شناسه‌ی داخلی ما). ترکیب `kind + tmdbId` یکتاست
`source`	`rule` : قانون نقض شد · `linked` : به یک مسدودی وصل است · `manual` : دستی
`label` و `reason`	نام و دلیل، تا بدون پرسیدن از TMDb خوانا باشد

- `ContentBlocksService` سه تابع دارد: `blocked(kind, ids)` (یک کوئری برای چند شناسه)، `isBlocked` و `without` (حذف آیتم‌های مسدود از یک آرایه).
- جدول با migration شماره‌ی `1793750400000` ساخته می‌شود. **باید روی دیتابیس برنامه اعمال شده باشد**، وگرنه خواندن آن خطای ۵۰۰ می‌دهد.

۵. ارتباط با کش Redis

- جستجوی فیلم، **نتیجه‌ی فیلترشده** را ۶ ساعت کش می‌کند و حکم «erotic» هر فیلم ۷ روز.
- لیست مسدودی **بعد از کشِ TMDb** اجرا می‌شود. خودِ لیست هم تا ۵ دقیقه در Redis (و ۳۰ ثانیه در حافظه) نگه داشته می‌شود تا هر درخواست به Neon نرود. اسکریپت ممیزی بعد از `--apply` این کش را پاک می‌کند؛ پس تغییر فوراً دیده می‌شود. اگر ردیفی را دستی به جدول اضافه کنید، حداکثر ۵ دقیقه طول می‌کشد.

- اگر **کد قانون‌ها** را عوض کنید، نتایج کش‌شده تا انقضا می‌مانند؛ برای پاک‌کردن، پیشوند `v1` را در `cache.service.ts` تغییر دهید.

۶. نکات عملی

می‌خواهم…	کار
یک فیلم یا فرد را دستی مسدود کنم	شناسه را به `MANUAL_*` در اسکریپت ممیزی اضافه کنید و اجرا کنید، یا یک ردیف با `source = 'manual'` در `content_block` بنویسید
فیلم بی‌گناهی را آزاد کنم	شناسه را به `KEEP_*` اضافه کنید و ردیف آن را از `content_block` پاک کنید
قانون‌ها را عوض کنم	تست‌ها را اجرا کنید (`content-filter.spec.ts`، `content-audit.spec.ts`، `movie-search.spec.ts`)، سپس ممیزی را دوباره اجرا کنید و کش را با تغییر `v1` پاک کنید
ببینم چه چیزی مسدود می‌شود	ممیزی را بدون `--apply` اجرا کنید و `audit/report.json` را بخوانید

> **هشدار**
>
> اجرای ممیزی با `--apply` رکوردهای دیتابیس را حذف می‌کند. اول روی یک شاخه‌ی Neon تمرین کنید، و آدرس مستقیم (بدون pooler) را در `AUDIT_DATABASE_URL` بگذارید.